

# Next-Gen AI Compiler

## Agents On The Control Plane

Agents Own Search · Orchestration · Synthesis.  
Compilers Own Lowering · Legality · Measure · Fallback.

**Ask: Is The Hybrid Prediction (~2027-31) The Right Bet For Your Roadmap?**

Expert Briefing · ~25-30 min + Q&A · Prediction-first

# Agenda

---

- 
1. Trends · Keep Substrate
  2. Verdict + Architecture
  3. Roadmap Checkpoints
  4. Commercial Blockers + Gaps
  5. Technical Prediction (Techniques)
  6. Stance · Ask · Appendix

# Trend backdrop — two stacks converging

---



**Compilers for AI**  
mature · fragmented substrate

**AI for compilers**  
rapid hybrid growth

Next-gen  $\neq$  throw away LLVM/MLIR — make them **agent-addressable**.

# Six active trends

---

## A — Hybrid guidance

Free IR rewrite fragile  
(mlirAgent < identity)  
Constrain actions: hints /  
advisory metadata / pass lists  
AgentCompile · HintPilot · LLM Com-  
piler

## B — RL → agents

CompilerGym → tool agents  
Heuristics as shippable C++  
Workflow compile · freeze · place  
Compiler-R1 · Magellan · FlowCompile

## C — MLIR + Triton

PyTorch → Inductor → Triton  
Also StableHLO → XLA / IREE  
Peak often vendor libs / Tile  
MLIR shared; product paths diverge

## D — Kernel agents

KernelBench: correct *and* faster  
One-shot often <20%; fusion hard  
GEAK · KernelLLM · AgentCompile  
Refine ≠ always faster

## E — Verify in the loop

Unit/golden → numerical  
→ Alive2-class local formal  
Weak on GPU races / FP noise  
Stacked oracle ladder for admit

## F — Broader object

Mid-decode diagnosis  
FMware: prompts / agents / knobs  
Agents as compiler engineers  
Compiler.next · Magellan · Claude C

# Keep substrate, change control plane

---

## Preserve

deterministic lowering  
library kernels  
build-CI regressability  
local formal checks

## Adopt

advisory search  
multi-agent orchestration  
heuristic synthesis  
profile / verifier feedback

**Anti-pattern: replace opt/Inductor  
with unconstrained LLM codegen**

# Executive verdict

---

Agents reshape the **control plane**  
more than they replace the **data plane**.

**Control plane:** search / orchestrate / synthesize

**Data plane:** lower / legality / measure / fallback

Compilers for AI  
× AI for compilers

Hybrid LLM-compiler loops

4th job Tier A:  
bring-up / codesign

## Will not happen soon

- Unconstrained LLM replaces Inductor/opt
- Autonomous chip tape-out via compiler agents (C10)

# Four agent jobs — architecture spine

---

## (a) Online

propose →  
measure  
admit

CompileIQ / GEAK  
ACCLAIM / HintPilot

## (b) Offline

evolve heuristics  
→ C++ / MLGO

Magellan  
AlphaEvolve

## (c) Eng.

oracle-gated  
pull request  
/ change

Archer

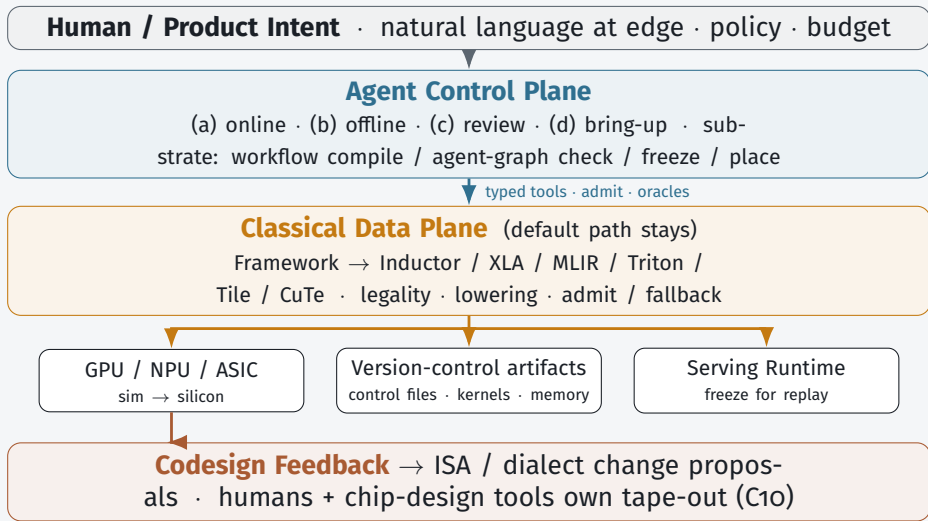
## (d) Bring-up

coverage →  
performance  
sim + silicon

TritorX  
KernelEvolve

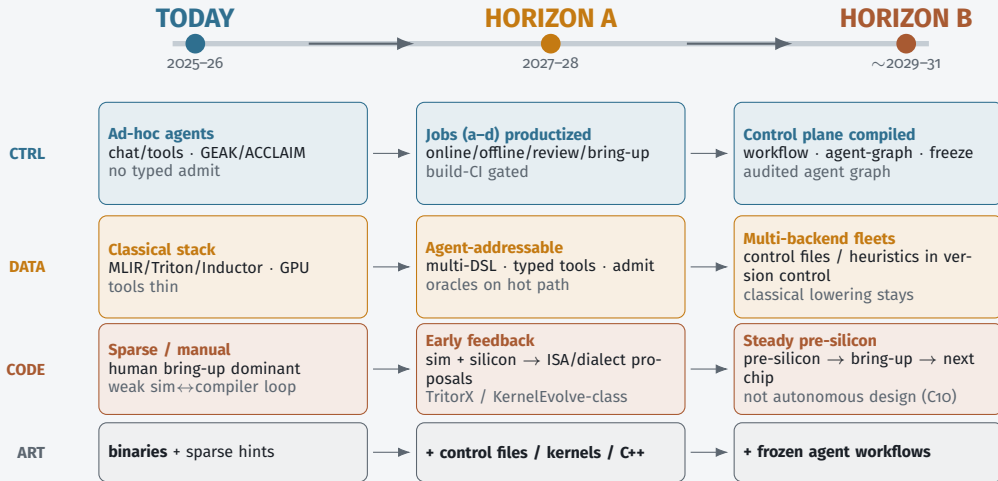
Control plane jobs sit *above* classical lowering — not instead of it.

# Architecture — Target Stack



Invariant: LLM guides search — does not silently define unchecked executable behavior.

# Architecture evolution — component changes



Components thicken left→right; data plane stays; agent graph becomes compiled & amortized.

# What ships / does not by 2028

---

## Predicted to ship

- Agent-addressable tool interfaces
- Hot-path specialize (not silent default-all)
- Magellan *and* MLGO both live
- Oracle pull-request review in serious orgs
- Coverage-first ASIC bring-up
- Triton-family primary; multi-DSL rising

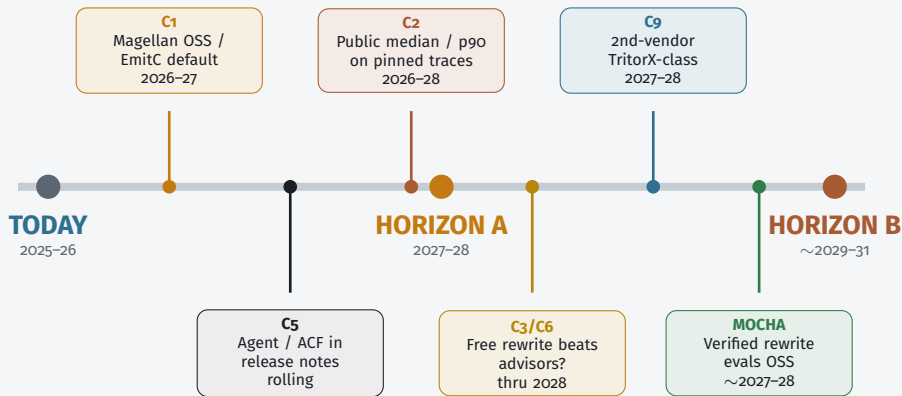
## Does not ship

- Unconstrained LLM replaces opt/Inductor
- One agent IR for all vendors
- Kernel agents uniformly beat eager on fusion ladders
- Autonomous microarch tape-out

Horizon A success = build-CI gated specialize + oracles + freeze artifacts

# Roadmap Checkpoints — when the prediction changes

---



Falsifiable milestones — watch settlement signals; do not average disagreements.

# Checkpoint C1 — heuristics vs neural advisors

---

## A — Magellan path

evolve shippable C++ heuristics

Evolve shippable C++  
(EVOLVE-BLOCK)

Offline coding agent  
is the control-plane output

Product = evolutionary coding agent +  
review

## B — MLGO / EmitC path

MLGO = learned in-tree LLVM advisors

NN advisors stay in-tree

June 2026 plan of record:

internal inliner →

Android/Fuchsia →

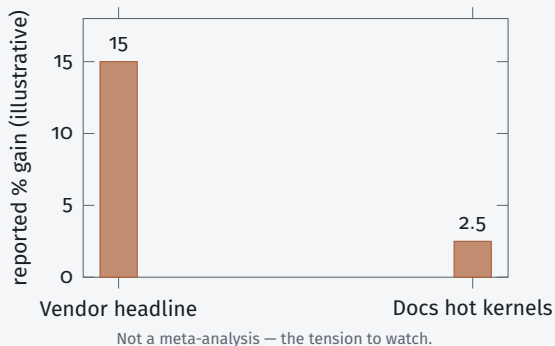
Chrome multi-model

Agents train/feature NNs

**Settlement:** public Magellan llvm patches that dis-  
place MLGO or EmitC-MLGO becomes customer default  
Watch LLVM syncs + OpenEvolve recipes quarterly through 2027

# Checkpoints C2 + C5 — gains & default path

---



**C2** “Agents become default” needs **median (p50) build-CI wins**, not best-kernel blogs.

p50 / p90 = 50th / 90th percentile of the gain distribution across builds.

**C5** Online specialize (control files / hints) vs offline eng (Magellan/MLGO) — both may live; *which is the default flag?*

**Settlement:** pinned public traces + release notes listing agent / control-file workflows.

# Checkpoints C<sub>3</sub> / C<sub>6</sub> / C<sub>9</sub> / C<sub>10</sub> — interface width & scope

---

## C<sub>3</sub> — free rewrite vs advisory

Flip: shared suite where free rewrite wins

⇒ wide rewrite interface vs narrow control files / hints

## C<sub>9</sub> — coverage vs peak

Flip: TritorX → KernelEvolve ladder public

⇒ SKU: coverage service target then performance target

## C<sub>6</sub> — replace vs control plane

Flip: production default with *no* admit

⇒ agents replaced the compiler (we reject this)

## C<sub>10</sub> — codesign vs auto tape-out

Flip: microarch primarily agent-proposed *and* compiler-oracle validated ⇒ enters chip-design tooling

# Commercial blockers — top 5

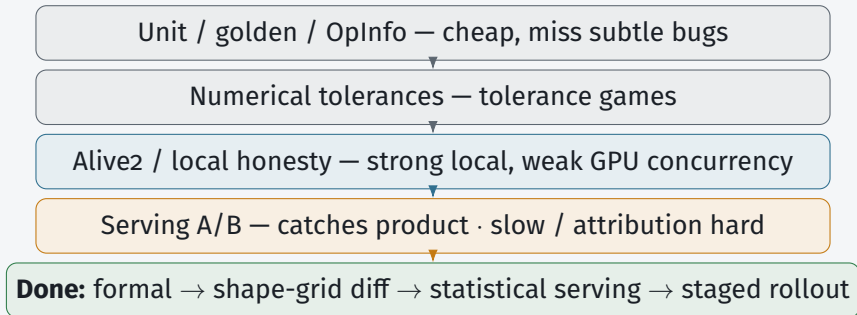
---

- 1 Oracles strong enough for money  
Fast-but-wrong · liability
- 2 Cost / replay / when agents run  
Nondeterministic \$N compiles
- 3 Agent $\leftrightarrow$ compiler contract  
Natural-language-only = demo
- 4 Distributional production evidence  
No default-path A/B
- 5 Ownership · supply chain · human review  
Unowned agent code in trusted base

Each blocker gets one slide — productization gaps, not research wishlist.

## Blocker 1 — Oracles for money

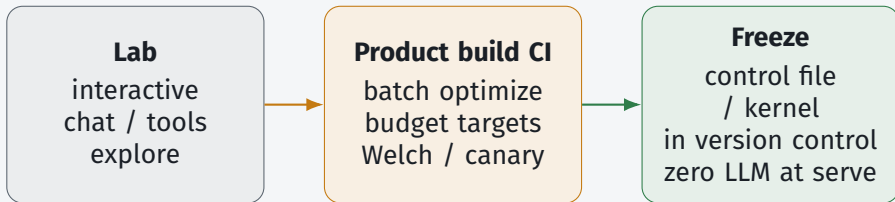
---



**Room question:** who owns false negatives when admit passes and production miscompiles?

## Blocker 2 — Cost, replay, when may the agent run?

---



Cache key: (IR hash, hardware, compiler ver, agent policy) · budget \$/%gain

Falsifies commercially: “chat with compiler on every build” without spend/latency targets.

## Blocker 3 — Agent $\leftrightarrow$ compiler contract

---

**A. Natural language + pasted logs** — demo only

**B. Structured admit traces** — build CI / bill of materials

**C. Typed tool interfaces** — required action space

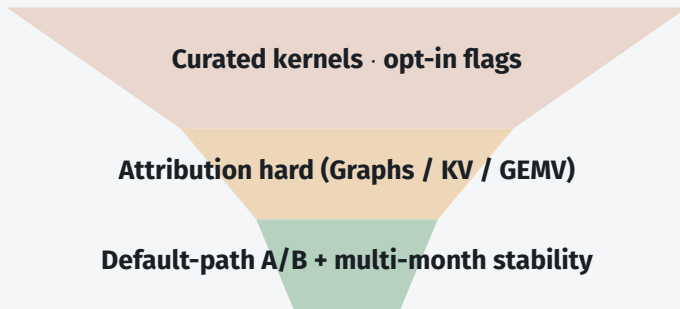
**D. Hybrid** — natural-language view over traces

```
admit_record = {graph_hash, hw_id, compiler_ver, action[], oracle[],  
artifact_digest, policy_id}
```

Lean: **C + B**; Natural language is a view, not source of truth. Advanced Control File = portable compiler knobs.

## Blocker 4 — Distributional production evidence

---



Marketing bar: median + 90th-percentile gains + cost-to-compile  
· persistent serving throughput, not single-kernel headlines

## Blocker 5 — Ownership, security, human review

---

### Trusted-base risk

agent kernels /  
heuristics enter  
trusted base

### Ownership

CODEOWNERS  
signed provenance  
sandbox

### Human-review capacity

agents  $\uparrow$  drafts  
reviewers =  
bottleneck

Lean: human CODEOWNER + signed admit + sand-  
box · oracle auto-merge only for **narrow** action classes  
Generic forge AI  $\neq$  compiler-oracle review (C7)

# Gap map — what blocks the prediction, not a wishlist

---

## HIGH

4.1 prod evidence

4.2 correctness

4.3 cost/replay

4.4 cross-stack

4.10 benchmarks

## MED-HIGH

4.5 hardware-native interfaces

4.7 training data

## MEDIUM

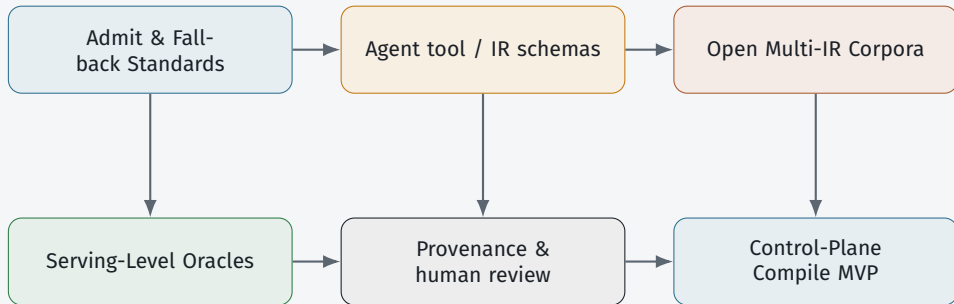
4.6 workflow compile

4.8 human-review process

4.9 security

# Cross-Cutting Research Agenda

---



Near-term research themes → next: technique map T1–T10 (exists / missing / unlocks).

# Technical Prediction — Accelerate The Roadmap

---

To settle checkpoints and ship Horizon A,  
enhance techniques *inside and outside* the compiler (T<sub>1</sub>–T<sub>10</sub>).

## Within (T<sub>1</sub>–T<sub>5</sub>)

- T1 Typed agent↔compiler interfaces
- T2 Admit / fallback machinery
- T3 Control files, hints, fingerprints + replay
- T4 Heuristic hooks & in-tree advisors
- T5 Dialect / ISA feedback sinks

## Outside (T<sub>6</sub>–T<sub>10</sub>)

- T6 Serving-level oracles & production A/B
- T7 Open multi-IR corpora (+ negatives)
- T8 Unified benchmark ladder
- T9 Provenance, ownership, HITL process
- T10 Agent-workflow compile / freeze / place

Enhancing only opt/Inductor/Triton internals is not enough. Next: exists · missing today · checkpoint unlocks.

# Technical Prediction — Within The Compiler (T<sub>I</sub>–T<sub>5</sub>)

---

Exists · Missing · Unlocks

## T<sub>1</sub> Typed interfaces

→ C<sub>3</sub>, C<sub>5</sub>, C<sub>6</sub>

Exists: CompileIQ · ACCLAIM · mlir-opt-repl · FlashInfer Trace

Missing: portable schemas across MLIR · Triton · Tile · StableHLO

## T<sub>2</sub> Admit / fallback

→ C<sub>6</sub> hybrid

Exists: AgentCompile · Archer · TritonRL verifiers · FlashInfer-Bench gates

Missing: shared admit *product* + trusted deterministic fallback

## T<sub>3</sub> Control files + replay

→ C<sub>2</sub>, C<sub>5</sub>

Exists: CompileIQ ACFs · FlashInfer Trace + `apply()`

Missing: content-addressed keys; golden replay on model upgrade

## T<sub>4</sub> Heuristic hooks / advisors

→ C<sub>1</sub>

Exists: Magellan / AlphaEvolve · MLGO · EmitC PoR

Missing: settled Magellan vs MLGO *default* on named apps

## T<sub>5</sub> Dialect / ISA sinks

→ C<sub>9</sub>, C<sub>10</sub>

Exists: TritonX · KernelEvolve · Ascend diagnosis

Missing: first-class change-proposal surfaces (not auto tape-out)

# Technical Prediction — Outside The Compiler (T6–T10)

---

Exists · Missing · Unlocks

## T6 Serving oracles / A/B

→ C2

Exists: unit/golden/Alive2 · **FlashInfer-Bench**+apply() · VibeServe judges

Missing: whole-program / GPU-race / FP contracts; multi-month default-path A/B

## T7 Multi-IR corpora

→ Selectors

Exists: Meta LLM Compiler · **KernelBook**→TritonRL · DR Triton synth

Missing: versioned MLIR / Tile / StableHLO + failed / miscompile negatives

## T8 Benchmark ladder

→ C2, C9

Exists: KernelBench(-X) · FlashInfer-Bench serving-kernel rung

Missing: full IR→kernel→fused→serving + cost-to-compile

## T9 Provenance / HITL

→ C7

Exists: Magellan reviewable C++ · Archer oracle review

Missing: CODEOWNERS + signed admit records + sandbox for proposals

## T10 Workflow compile / freeze

→ Horiz. B

Exists: FlowCompile · Auto · AgentFlow · VibeServe (early)

Missing: shared agent-graph IR + fail-closed CI (productize → Horizon B)

# Technical Prediction — What Each Checkpoint Needs

---

**C1** ← **T4** Magellan-class synthesis *or* EmitC-MLGO customer default on named apps.

**C2** ← **T3+T6+T8** replayable artifacts + serving oracles + ladder; FlashInfer-Bench = pressure, not settlement.

**C5** ← **T1+T3** typed interfaces + freeze-before-serve named in release notes.

**C3 / C6** ← **T1+T2** constrained tools + admit/fallback (lean: hybrid advisory until free rewrite proves out).

**C9** ← **T5+T8** coverage→perf agents + dialect sinks; need second-vendor TritonX-class path.

**C10** ← **T5** failure modes → ISA/dialect *proposals only*; humans + chip-design tools own tape-out.

# Technical Prediction — Critical Missing Parts Now

---

## Highest-leverage bets to accelerate Horizon A

### 1. Money-grade oracle stack (T2+T6)

local formal → shape-grid diff  
→ serving statistics → staged rollout  
Without this, agents stay demos

### 2. Replayable artifact contract (T3)

control files / kernels / heuristics  
with cache keys + golden replay  
Without this, CI rejects the loop

### 3. Portable agent compile interface (T1)

summaries · actions · admit records  
across MLIR / Triton / Tile / StableHLO  
Without this, every stack re-glues

### 4. Open ladder + multi-IR data (T7+T8)

correctness × speed × \$/compile  
plus negative (failed) examples  
Without this, claims stay incomparable

Survey lean: T1–T5 must ship as product surfaces; T6–T10 are equally first-class — mostly outside classical lowering.

# Org Adoption Questions

---

1. Online (CompileIQ) vs Offline (Magellan) — which matches your release model?
2. Oracle stack: Alive2 / golden / serving A/B — who owns false negatives?
3. Agent contract surface: LLVM / MLIR / Triton / StableHLO / Tile?
4. How do you cache/regress traces across compiler *and* model upgrades?
5. Max \$/build for a median X% win? Named maintainer per admitted artifact?

## Working stance until conflicts settle

---

- 1 Hybrid: agents search; compilers lower (not replace)
- 2 Magellan || MLGO — parallel bets (C1)
- 3 Discount single-number speedups (C2)
- 4 Constrained actions + strong oracles (C3)
- 5 Demote generic SCM AI (C7)
- 6 Coverage→performance ladder; no autonomous chip design (C9/C10)

# Commercial checklist — handout

---

## Architecture

typed tools + admit  
traces  
version control  $\gg$  dense  
memory  $\gg$  scratchpad  
state-machine plan for  
service targets  
freeze before default-on

## Trust

layered oracles  
CODEOWNERS + prove-  
nance  
joint version pins  
A/B before “prod default”

## Business

sell regressable artifacts  
+ build-CI quota  
customer owns outputs  
coverage then perfor-  
mance service target  
token budget per %gain

# Discussion

Thank you — hybrid bet stays the ask

1. Which checkpoint flips your investment first — C1, C2, or C9?
2. Harder commercial bar: **oracle strength** or **replay/freeze**?
3. Building job (a), (b), or (d) first — what do you refuse online?

Ask again: is the hybrid bet right for your roadmap? Watch settlement signals — do not average disagreements.

# Appendix

## Evidence Store Snapshot

Publications digests · commercial product signals · open repositories.  
Tiered for the agentic-compiler prediction — not a full catalog.

---

# Appendix — evidence map

---

## Publications

~115 digests  
one file per source  
papers · blogs · talks  
forge pages · minutes  
★ = prediction-critical

## Products

Commercial SKUs as  
*prediction signals*  
Tier A shapes the future  
Tier B = data-plane de-  
faults  
not a full SKU list

## Repositories

GitHub / Gerrit / forge  
artifacts tiered A/B/C  
A = agents reshape com-  
pile  
C = generic forge AI only  
not an exhaustive forge  
map

Use ★ digests + Tier A products/repos when a checkpoint or blocker is contested.

# Appendix — Tier A commercial signals

---

Company offerings that shape what ships — roles: agent loop / kernel surface / cloud agent

**Google / DeepMind** — AlphaEvolve Cloud (GA); Magellan + MLGO (offline heuristics *and* neural advisors)

**NVIDIA** — CompileIQ (+ agent-skills); CUDA Tile / Tile IR; TensorRT-LLM agent skills

**AMD** — GEAk (multi-agent Triton / Instinct kernel optimization)

**Meta** — LLM Compiler / KernelLLM; TritonX + KernelEvolve (ASIC bring-up); Helion

**FlashInfer / NVIDIA · UW · CMU** — FlashInfer-Bench (serving-trace ladder + apply() into SGLang/vLLM)

**Tier B baselines:** TensorRT-LLM · Inductor · XLA/StableHLO · FlashInfer kernels · Modular MAX · OpenVINO · Neuron · Hexagon-MLIR

# Appendix — Tier A open repositories

---

## Offline / review / heuristics

OpenEvolve (Magellan-class loop)  
HeuriGym (heuristic bench)  
Archer (oracle pull-request review)  
Compiler-R1 · HintPilot  
mlirAgent (negative free-rewrite)

## Online / kernels / bring-up

ACCLAIM · CompileIQ  
GEAK · KernelAgent  
KernelBench / KernelBench-X  
FlashInfer-Bench · AutoKernel  
Helion · TritonX / KernelEvolve

**How to read tiers:** A = agents change heuristics/kernels/knobs/review with domain oracles. B = data-plane hosts. C = generic forge AI (demoted for prediction).

# Appendix — prediction-critical digests (★)

---

Highest-signal sources for the hybrid prediction — mechanisms, not headlines

**Vision / funded:** Compiler 2.0 Ken Kennedy plenary (MIT) · DARPA MOCHA / Aarno Compiler 2.0

**Offline / online jobs:** Magellan (+ LLVM slides) · ACCLAIM (+ GitHub) · EmitC-MLGO June 2026 plan-of-record minutes

**Bring-up / codesign:** TritorX · KernelEvolve · Ascend hierarchical diagnosis · KForge

**Kernels / ladder / data:** CompileIQ agent-skills · FlashInfer-Bench ★ · KernelBook ★ · AutoKernel · CuTeGen · Auto · Flow-Compile

Full digest index: one file per source under the publications store (~115 entries). Technique map: SURVEY §5.8 T1–T10.

# Appendix — publication groups

---

**17** GPU kernels & inference  
**13** Agentic & RL compilers  
**13** Source control & review

**10** Classic DL compilers  
**10** Company infra  
**9** Forums & workshops

**8** Surveys & vision  
**8** Foundation LLMs  
**6** MLGO & RL gyms

**5** HW codesign & bring-up  
**4** Commercial products  
**4** Control-plane substrate

## How digests are used

Mechanism first; demote generic forge AI.

When sources disagree, keep both sides — do not average.

Update prediction only when Tier A / ★ evidence moves.